Lecture 14: Time Stepping: Euler, Leapfrog, Trapezoidal

18.085 Computational Science and Engineering I • MW 9:30–11, 6-120 • Monday, November 2, 2026

After this lecture you should be able to

- Derive forward Euler, backward Euler, leapfrog and the trapezoidal method from Taylor series or from a rule for the integral of $\boldsymbol{f}$ over one step.
- Write each method as a growth matrix $\boldsymbol{G}$ for the oscillator $\ddot{u} + u = 0$.
- Decide stability from the eigenvalues of $\boldsymbol{G}$, and read off what each method does to the energy and to the phase.
- Find the step limit $h < 2$ of leapfrog and the quantity it conserves.
- Draw each method as an area under the curve of $\boldsymbol{f}$, and read off what one step misses.

1 Forward Euler, from Lecture 13

We begin with the derivation of forward Euler from last time, as it was given there.

The mass–spring system is also a good toy problem for touching upon *finite difference methods for time-based differential equations*, also known as **dynamical systems**.

For simplicity, let us shrink all the way down to one mass and one spring. The system is

$$\ddot{u} + u = 0,$$

which is a *second-order* system.

First insight: lower the order by adding variables

We can lower the order of the system by introducing new variables. Define

$$v \equiv \dot{u}, \qquad \text{i.e.} \qquad \frac{du}{dt} = v,$$

and then $\ddot{u} + u = 0$ says exactly

$$\frac{dv}{dt} = -u.$$

Taken together we now have *only first-order* time derivatives:

$$\frac{d}{dt}\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}\begin{bmatrix} u \\ v \end{bmatrix}.$$

Interpretation

v is the velocity of the mass on the spring. Instead of a single second-order equation in the displacement u alone, we are studying an equivalent *system* of first-order equations in u and v.

In general, a system of first-order differential equations with respect to time is called a **dynamical system**. In principle any kind of higher-order time dynamics can be reduced to a dynamical system with only first derivatives in time, through the introduction of new variables such as v.

One canonical form

Order is not a fundamental property of a time-dependent problem: it is a choice of variables. Reduce everything to $\dot{\boldsymbol{y}} = \boldsymbol{A}\boldsymbol{y}$ and you only ever need *one* family of time-stepping methods, applied to a vector.

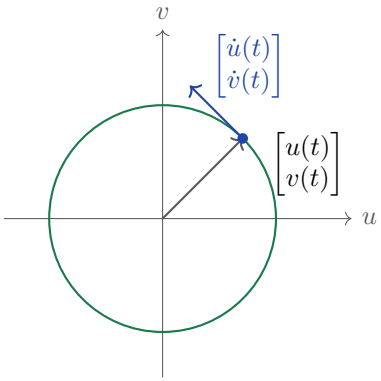
Phase space

To visualise the time evolution of dynamical systems we often plot the trajectories of the dependent variables in a **phase space diagram**. "Phase space" is simply a technical term for the possible values the dependent variables can take on.

For the mass–spring system, $\dot{u} = v$ and $\dot{v} = -u$. Note that

$$\begin{bmatrix} \dot{u} & \dot{v} \end{bmatrix} \cdot \begin{bmatrix} u \\ v \end{bmatrix} = \dot{u}\,u + \dot{v}\,v = (v)\,u + (-u)\,v = uv - uv = 0.$$

So the time derivative of the vector $\begin{bmatrix} u \\ v \end{bmatrix}$ is *always orthogonal to it.* A vector whose rate of change is perpendicular to itself never changes length. Therefore the phase-space trajectory $\begin{bmatrix} u(t) \\ v(t) \end{bmatrix}$ is a **circle**.



The circle is conservation of energy

The radius $\sqrt{u^2 + v^2}$ is fixed by the energy in the system, which the true dynamics conserve. Everything interesting about the numerical methods below is a statement about what they do to that radius.

Discretising time

Suppose we now discretise time. Put

$$t_n = n\,\Delta t, \qquad U_n \equiv u(t_n), \qquad V_n \equiv v(t_n),$$

and replace each time derivative by a forward difference,

$$\left.\frac{du}{dt}\right|_{t_n} \simeq \frac{U_{n+1} - U_n}{\Delta t}, \qquad \left.\frac{dv}{dt}\right|_{t_n} \simeq \frac{V_{n+1} - V_n}{\Delta t}.$$

So the two equations of the dynamical system become

$$\frac{U_{n+1}-U_n}{\Delta t} = v, \qquad \frac{V_{n+1}-V_n}{\Delta t} = -u.$$

The whole question in one line

What grid points should we use on the right-hand side? The left-hand sides are settled; the right-hand sides still say u and v with no subscript. Different choices of which time level to evaluate them at lead to different methods, and to completely different long-time behaviour.

Forward Euler

Take everything on the right at the *old* time level:

$$\text{RHS} = \begin{bmatrix} V_n \\ -U_n \end{bmatrix}.$$

We obtain

$$\begin{cases} \dfrac{U_{n+1}-U_n}{\Delta t} = V_n \\ \dfrac{V_{n+1}-V_n}{\Delta t} = -U_n \end{cases} \quad \Longrightarrow \quad \begin{cases} U_{n+1} = U_n + \Delta t\, V_n \\ V_{n+1} = V_n - \Delta t\, U_n \end{cases}$$

which is *explicit*: the new values are written down directly from the old ones. In matrix form,

$$\begin{bmatrix} U_{n+1} \\ V_{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & \Delta t \\ -\Delta t & 1 \end{bmatrix}}_{\boldsymbol{G}_F} \begin{bmatrix} U_n \\ V_n \end{bmatrix},$$

and $\boldsymbol{G}_F$ is called the **growth matrix**. The end result of n steps is

$$\begin{bmatrix} U_n \\ V_n \end{bmatrix} = \boldsymbol{G}_F^n \begin{bmatrix} U_0 \\ V_0 \end{bmatrix}, \qquad \text{with } \begin{bmatrix} U_0 \\ V_0 \end{bmatrix} \text{ the initial conditions.}$$

Qualitatively, what happens to $\begin{bmatrix} U_n \\ V_n \end{bmatrix}$ as n gets large depends on the *eigenvalues* of $\boldsymbol{G}_F$. Writing $h = \Delta t$ for brevity, $\boldsymbol{G}_F$ has eigenvalues

$$\lambda = 1 + ih, \qquad \lambda = 1 - ih, \qquad \text{so} \qquad |\lambda| = \sqrt{1+h^2} > 1,$$

and we will see a *growth* in the size of the vector $\begin{bmatrix} U_n \\ V_n \end{bmatrix}$.

What we compare

Write $\boldsymbol{Y}_n = (U_n, V_n)$ for the computed vector at $t_n = nh$, which approximates $\boldsymbol{y}(t_n) = (u(t_n), v(t_n))$, and $\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right]$, so that the oscillator is $\dot{\boldsymbol{y}} = \boldsymbol{A}\boldsymbol{y}$. Forward Euler is $\boldsymbol{Y}_{n+1} = \boldsymbol{G}_F \boldsymbol{Y}_n$ with $\boldsymbol{G}_F = \boldsymbol{I} + h\boldsymbol{A}$.

Over one step of length h the exact solution of the oscillator is a rotation,

$$\boldsymbol{y}(t+h) = \begin{bmatrix} \cos h & \sin h \\ -\sin h & \cos h \end{bmatrix} \boldsymbol{y}(t),$$

which turns the point clockwise through the angle h and keeps its length; its eigenvalues are $e^{\pm ih}$. Every method in this lecture replaces the rotation by a **growth matrix** $\boldsymbol{G}$, with $\boldsymbol{Y}_{n+1} = \boldsymbol{G}\boldsymbol{Y}_n$ and $\boldsymbol{Y}_n = \boldsymbol{G}^n\boldsymbol{Y}_0$.

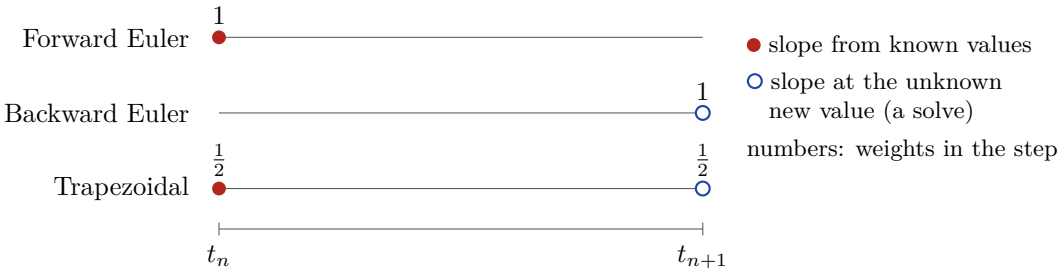
Two questions for every method

Write the eigenvalues of $\boldsymbol{G}$ as $\lambda = |\lambda|\, e^{\pm i\theta}$.

- *Amplitude.* Is $|\lambda| = 1$? If $|\lambda| > 1$ the computed oscillation grows; if $|\lambda| < 1$ it dies away.
- *Phase.* Is $\theta = h$? If $\theta < h$ the computed oscillation falls behind the true one; if $\theta > h$ it runs ahead.

The true rotation has $|\lambda| = 1$ and $\theta = h$.

The methods differ in where, inside the step from t_n to t_{n+1}, they evaluate the right-hand side. Leapfrog staggers its two variables and gets its own picture in Section 3; the other three are here.



A closer look at forward Euler

How accurate is the forward difference?

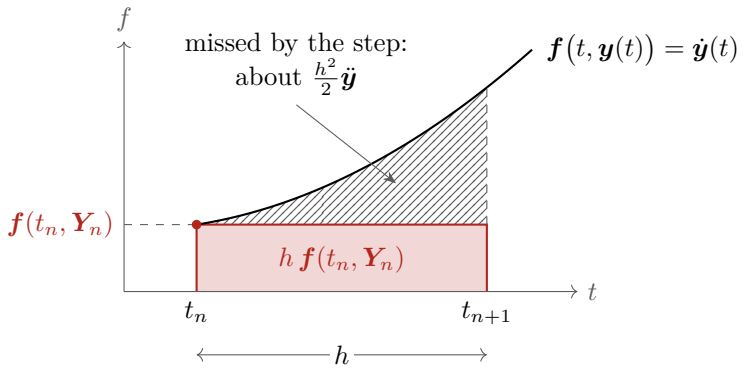
Taylor's theorem gives

$$u(t_{n+1}) = u(t_n) + h\,\dot{u}(t_n) + \frac{h^2}{2}\,\ddot{u}(t_n) + \cdots, \qquad \text{so} \qquad \frac{u(t_{n+1}) - u(t_n)}{h} = \dot{u}(t_n) + \frac{h}{2}\,\ddot{u}(t_n) + \cdots.$$

The forward difference approximates the derivative at the old time t_n with an error of order h. Each step misses the term $\frac{h^2}{2}\ddot{\boldsymbol{y}}$; over a fixed time T there are T/h steps, so these errors add up to order h: forward Euler is a **first-order** method. For any system $\dot{\boldsymbol{y}} = \boldsymbol{f}(t, \boldsymbol{y})$ the method reads $\boldsymbol{Y}_{n+1} = \boldsymbol{Y}_n + h\,\boldsymbol{f}(t_n, \boldsymbol{Y}_n)$. The same formula comes from the integral form

$$\boldsymbol{y}(t_{n+1}) = \boldsymbol{y}(t_n) + \int_{t_n}^{t_{n+1}} \boldsymbol{f}\big(t, \boldsymbol{y}(t)\big)\, dt,$$

if the integral is replaced by a rectangle of width h whose height $\boldsymbol{f}(t_n, \boldsymbol{Y}_n)$ is taken at the left end (sketch below). The other methods of this lecture are other rules for the same integral, except leapfrog, which comes from the second-order equation itself.



Forward Euler as a rule for the integral. The area under the curve from t_n to t_{n+1} is the exact change $\boldsymbol{y}(t_{n+1}) - \boldsymbol{y}(t_n)$. Forward Euler keeps the shaded rectangle, of width h and height $\boldsymbol{f}(t_n, \boldsymbol{Y}_n)$ taken at the left end (drawn here for $\boldsymbol{Y}_n = \boldsymbol{y}(t_n)$). The hatched piece above it is what one step misses: near t_n the curve rises at the rate $\ddot{\boldsymbol{y}}$, so the piece is close to a triangle of base h and height $h\,\ddot{\boldsymbol{y}}$, with area $\frac{h^2}{2}\ddot{\boldsymbol{y}}$, the term found from Taylor's theorem above.

Eigenvalues and energy

The eigenvalues come from

$$\det(\boldsymbol{G}_F - \lambda \boldsymbol{I}) = (1-\lambda)^2 + h^2 = 0 \qquad \Longrightarrow \qquad 1 - \lambda = \mp ih \qquad \Longrightarrow \qquad \lambda = 1 \pm ih,$$

so $|\lambda| = \sqrt{1+h^2} > 1$ and the angle per step is $\theta = \arctan h$. The energy can be followed directly. The cross terms cancel in

$$U_{n+1}^2 + V_{n+1}^2 = (U_n + hV_n)^2 + (V_n - hU_n)^2 = (1+h^2)(U_n^2 + V_n^2),$$

so each step multiplies the energy by $1 + h^2$, whatever the starting point (the red spiral in the figure of Section 2 shows it over one period). Geometrically, the step $h\,(V_n, -U_n)$ is perpendicular to (U_n, V_n) and h times as long, so by Pythagoras the new point is $\sqrt{1+h^2}$ times as far from the origin as the old one (left panel of the figure in Section 2). In matrix terms $\boldsymbol{G}_F^{\mathsf{T}}\boldsymbol{G}_F = (1+h^2)\,\boldsymbol{I}$: $\boldsymbol{G}_F$ is $\sqrt{1+h^2}$ times a rotation through $\arctan h = h - h^3/3 + \cdots$, an angle a little smaller than h. Over one period, N steps of $h = 2\pi/N$, the amplitude grows by

$$(1+h^2)^{N/2} = \Big(1 + \frac{4\pi^2}{N^2}\Big)^{N/2} \approx e^{2\pi^2/N}.$$

The factor is 2.22 for $N = 24$ steps per period (the exponential gives 2.28) and still 1.02 for $N = 1000$.

Example 1.1 (Forward Euler by hand)**. Given.** $\ddot{u} + u = 0$ with $u(0) = 1$, $v(0) = 0$, so $\boldsymbol{Y}_0 = (1, 0)$, and a step h.

Find. What are $\boldsymbol{Y}_1 = \boldsymbol{G}_F\boldsymbol{Y}_0$ and $\boldsymbol{Y}_2 = \boldsymbol{G}_F^2\boldsymbol{Y}_0$ for a general h? What are they for $h = 0.5$? What happens to $U_n^2 + V_n^2$, and how far is $\boldsymbol{Y}_2$ from the true point at $t = 1$?

Answer. Keep h as a letter first:

$$\boldsymbol{Y}_1 = \boldsymbol{G}_F\boldsymbol{Y}_0 = \begin{bmatrix} 1 & h \\ -h & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -h \end{bmatrix}, \qquad \boldsymbol{G}_F^2 = \begin{bmatrix} 1 & h \\ -h & 1 \end{bmatrix}\begin{bmatrix} 1 & h \\ -h & 1 \end{bmatrix} = \begin{bmatrix} 1-h^2 & 2h \\ -2h & 1-h^2 \end{bmatrix},$$

$$\boldsymbol{Y}_2 = \boldsymbol{G}_F\boldsymbol{Y}_1 = \begin{bmatrix} 1 + h\,(-h) \\ -h + 1\cdot(-h) \end{bmatrix} = \begin{bmatrix} 1-h^2 \\ -2h \end{bmatrix}, \quad \text{the same as} \quad \boldsymbol{Y}_2 = \boldsymbol{G}_F^2\boldsymbol{Y}_0 = \begin{bmatrix} 1-h^2 \\ -2h \end{bmatrix}.$$

The squared lengths are $|\boldsymbol{Y}_1|^2 = 1 + h^2$ and $|\boldsymbol{Y}_2|^2 = (1-h^2)^2 + 4h^2 = 1 + 2h^2 + h^4 = (1+h^2)^2$, as $\boldsymbol{G}_F^{\mathsf{T}}\boldsymbol{G}_F = (1+h^2)\,\boldsymbol{I}$ says. Now put $h = 0.5$:

$$\boldsymbol{G}_F = \begin{bmatrix} 1 & 0.5 \\ -0.5 & 1 \end{bmatrix}, \qquad \boldsymbol{Y}_1 = \begin{bmatrix} 1 \\ -0.5 \end{bmatrix}, \qquad \boldsymbol{Y}_2 = \begin{bmatrix} 1 - 0.25 \\ -2 \cdot 0.5 \end{bmatrix} = \begin{bmatrix} 0.75 \\ -1 \end{bmatrix},$$

so $(U_1, V_1) = (1, -0.5)$ and $(U_2, V_2) = (0.75, -1)$. $U_n^2 + V_n^2$ goes $1, 1.25, 1.5625$: it is multiplied by $1 + h^2 = 1.25$ at each step. The true point is $(\cos 1, -\sin 1) = (0.540, -0.841)$, so after two steps the computed point is off by 0.263, about a quarter of the radius.

2 Backward Euler

The same start as forward Euler

As for forward Euler, put

$$t_n = n\,h, \qquad U_n \approx u(t_n), \qquad V_n \approx v(t_n), \qquad \boldsymbol{Y}_n = \begin{bmatrix} U_n \\ V_n \end{bmatrix},$$

where U_n and V_n are the computed values (Lecture 13 wrote $U_n \equiv u(t_n)$), and replace each time derivative by the same difference quotient,

$$\frac{du}{dt} \simeq \frac{U_{n+1} - U_n}{h}, \qquad \frac{dv}{dt} \simeq \frac{V_{n+1} - V_n}{h}.$$

This time read the quotient as the derivative at the *new* time t_{n+1}. Taylor's theorem about t_{n+1} gives

$$u(t_n) = u(t_{n+1}) - h\,\dot{u}(t_{n+1}) + \frac{h^2}{2}\,\ddot{u}(t_{n+1}) - \cdots,$$

so that

$$\frac{u(t_{n+1}) - u(t_n)}{h} = \dot{u}(t_{n+1}) - \frac{h}{2}\,\ddot{u}(t_{n+1}) + \cdots,$$

the same accuracy, order h, as before. As in Section 1 the two equations of the dynamical system become

$$\frac{U_{n+1} - U_n}{h} = v, \qquad \frac{V_{n+1} - V_n}{h} = -u,$$

and the question is again which time level to use on the right-hand side.

The right-hand side at the new time level

Now take everything on the right at the *new* time level:

$$\text{RHS} = \begin{bmatrix} V_{n+1} \\ -U_{n+1} \end{bmatrix}.$$

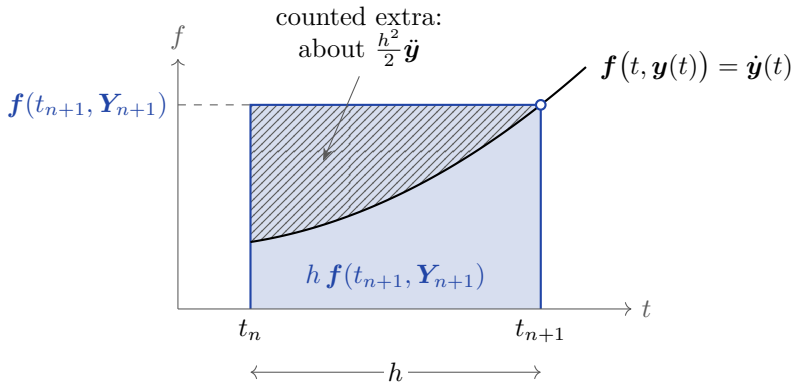
We obtain

$$\begin{cases} \dfrac{U_{n+1} - U_n}{h} = V_{n+1} \\ \dfrac{V_{n+1} - V_n}{h} = -U_{n+1} \end{cases} \qquad \Longrightarrow \qquad \begin{cases} U_{n+1} = U_n + h\,V_{n+1} \\ V_{n+1} = V_n - h\,U_{n+1} \end{cases}$$

The unknowns now appear on both sides, so we gather them on the left:

$$\begin{cases} U_{n+1} - h\,V_{n+1} = U_n \\ h\,U_{n+1} + V_{n+1} = V_n \end{cases} \quad\Longrightarrow\quad \begin{bmatrix} 1 & -h \\ h & 1 \end{bmatrix} \begin{bmatrix} U_{n+1} \\ V_{n+1} \end{bmatrix} = \begin{bmatrix} U_n \\ V_n \end{bmatrix}, \quad \text{that is} \quad (\boldsymbol{I} - h\boldsymbol{A})\,\boldsymbol{Y}_{n+1} = \boldsymbol{Y}_n.$$

That is an *implicit* method: each step costs a solve. For a general system $\dot{\boldsymbol{y}} = \boldsymbol{f}(t, \boldsymbol{y})$ backward Euler reads $\boldsymbol{Y}_{n+1} = \boldsymbol{Y}_n + h\,\boldsymbol{f}(t_{n+1}, \boldsymbol{Y}_{n+1})$: in the integral form, the height of the rectangle is taken at the right end (sketch below). For a nonlinear $\boldsymbol{f}$ each step is a nonlinear system, usually solved by Newton's method starting from $\boldsymbol{Y}_n$; each Newton step solves a linear system with the matrix $\boldsymbol{I} - h\boldsymbol{J}$, where $\boldsymbol{J} = \partial \boldsymbol{f}/\partial \boldsymbol{y}$. For $\boldsymbol{f} = \boldsymbol{A}\boldsymbol{y}$ the Jacobian is $\boldsymbol{A}$ and one linear solve is enough.



Backward Euler as a rule for the integral, on the same curve as the forward Euler sketch of Section 1. The rectangle takes its height $\boldsymbol{f}(t_{n+1}, \boldsymbol{Y}_{n+1})$ at the right end, at the new point that the step solves for (open circle, drawn here at the true value $\boldsymbol{Y}_{n+1} = \boldsymbol{y}(t_{n+1})$). For a rising curve it covers the whole area under the curve and the hatched piece above it as well: it counts about $\frac{h^2}{2}\ddot{\boldsymbol{y}}$ too much, the size of the piece that forward Euler misses, with the opposite sign.

Solving for the new values

Solve the 2×2 system by elimination. The second equation gives $V_{n+1} = V_n - hU_{n+1}$; put it into the first:

$$U_{n+1} - h(V_n - hU_{n+1}) = U_n \quad\Longrightarrow\quad (1 + h^2)\,U_{n+1} = U_n + hV_n \quad\Longrightarrow\quad U_{n+1} = \frac{U_n + hV_n}{1 + h^2}.$$

Then back-substitute:

$$V_{n+1} = V_n - h\,\frac{U_n + hV_n}{1 + h^2} = \frac{(1 + h^2)\,V_n - hU_n - h^2V_n}{1 + h^2} = \frac{V_n - hU_n}{1 + h^2}.$$

So

$$\boldsymbol{G}_B = (\boldsymbol{I} - h\boldsymbol{A})^{-1} = \frac{1}{1 + h^2} \begin{bmatrix} 1 & h \\ -h & 1 \end{bmatrix} = \frac{\boldsymbol{G}_F}{1 + h^2},$$

and multiplying back, $\begin{bmatrix} 1 & -h \\ h & 1 \end{bmatrix} \boldsymbol{G}_B = \boldsymbol{I}$ confirms it. The backward Euler point is the forward Euler point pulled in towards the origin by the factor $1/(1 + h^2)$.

Eigenvalues and energy

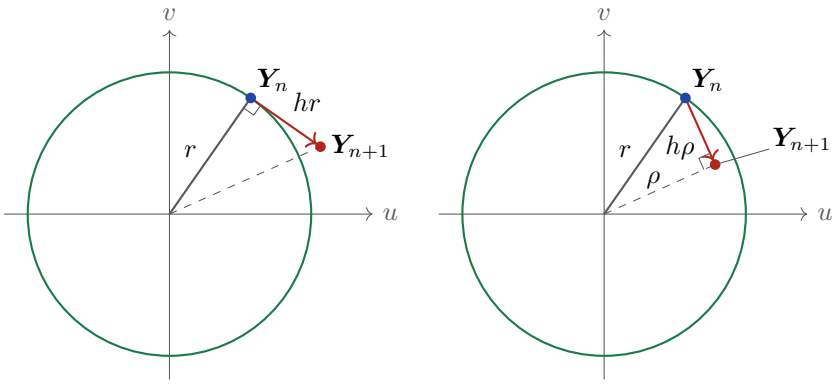
The eigenvalues of $\boldsymbol{I} - h\boldsymbol{A}$ are $1 \mp ih$, and $\boldsymbol{G}_B$ is its inverse, so its eigenvalues are the reciprocals:

$$\lambda = \frac{1}{1 \mp ih} = \frac{1 \pm ih}{1 + h^2}, \qquad |\lambda| = \frac{1}{\sqrt{1 + h^2}} < 1, \qquad \theta = \arctan h,$$

the same angle as forward Euler. For the energy, write the old point in terms of the new one, $U_n = U_{n+1} - hV_{n+1}$ and $V_n = V_{n+1} + hU_{n+1}$. The cross terms cancel as before:

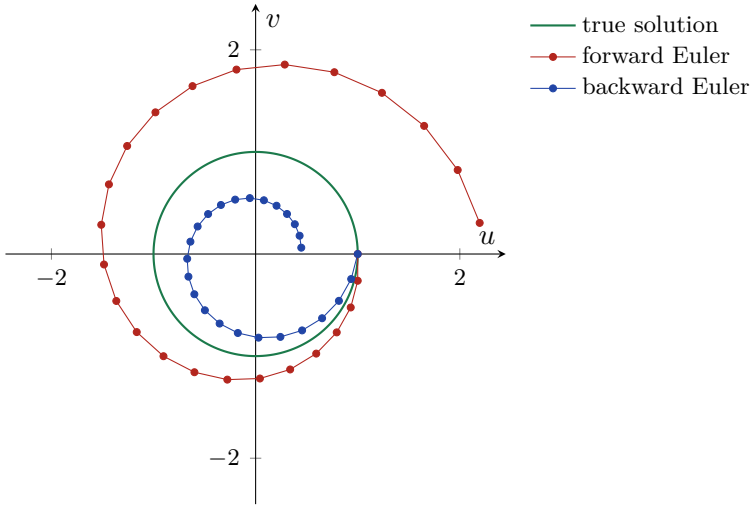
$$U_n^2 + V_n^2 = (U_{n+1} - hV_{n+1})^2 + (V_{n+1} + hU_{n+1})^2 = (1 + h^2)(U_{n+1}^2 + V_{n+1}^2).$$

So each step divides the energy by $1 + h^2$, and we end up with a *decay*. The geometry is that of forward Euler run backwards: the step is at right angles to the *new* radius, the line from the origin to the new point, so by Pythagoras the old point is the one farther from the origin.



forward Euler backward Euler

One step from the same point $\boldsymbol{Y}_n$ on the circle of radius r, with $h = 0.6$. Forward Euler steps along the tangent at the old point and lands outside the circle, at the distance $r\sqrt{1 + h^2}$ from the origin. Backward Euler steps at right angles to the new radius and lands inside, at the distance ρ with $\rho^2 + h^2\rho^2 = r^2$, that is $\rho = r/\sqrt{1 + h^2}$. The two new points lie on the same ray from the origin.



Forward and backward Euler over one period, 24 steps per period, from $(u, v) = (1, 0)$. The green circle has radius 1. After one period the radius is 2.22 and 0.45.

Decaying problems, and the cost

For a decaying problem $\dot{y} = -ay$ with $a > 0$, backward Euler gives $Y_{n+1} = Y_n - ah\,Y_{n+1}$, so $Y_{n+1} = Y_n/(1+ah)$: the computed solution decays at every step size. Forward Euler gives $Y_{n+1} = (1-ah)\,Y_n$, which grows in size once $ah > 2$. Lecture 15 compares all the methods in this way. The price of backward Euler is the solve with $\boldsymbol{I} - h\boldsymbol{A}$ at every step. For a large system with m unknowns and a fixed h, factor $\boldsymbol{I} - h\boldsymbol{A} = \boldsymbol{LU}$ once (Lecture 3); each step is then a forward and a back substitution.

Example 2.1 (Backward Euler by hand)**. Given.** The same oscillator and start as in Example 1.1, $\boldsymbol{Y}_0 = (1, 0)$, and a step h.

Find. What are $\boldsymbol{Y}_1 = \boldsymbol{G}_B\boldsymbol{Y}_0$ and $\boldsymbol{Y}_2 = \boldsymbol{G}_B^2\boldsymbol{Y}_0$ for a general h, and for $h = 0.5$? What happens to $U_n^2 + V_n^2$, and how far is $\boldsymbol{Y}_2$ from the true point at $t = 1$?

Answer. With h as a letter,

$$\boldsymbol{Y}_1 = \frac{1}{1+h^2}\begin{bmatrix} 1 & h \\ -h & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{1+h^2}\begin{bmatrix} 1 \\ -h \end{bmatrix}, \qquad \boldsymbol{Y}_2 = \frac{\boldsymbol{G}_F^2\boldsymbol{Y}_0}{(1+h^2)^2} = \frac{1}{(1+h^2)^2}\begin{bmatrix} 1-h^2 \\ -2h \end{bmatrix},$$

using $\boldsymbol{G}_B = \boldsymbol{G}_F/(1+h^2)$ and $\boldsymbol{G}_F^2\boldsymbol{Y}_0$ from Example 1.1. Now put $h = 0.5$, so $1 + h^2 = 1.25$:

$$\boldsymbol{G}_B = \frac{1}{1.25}\begin{bmatrix} 1 & 0.5 \\ -0.5 & 1 \end{bmatrix} = \begin{bmatrix} 0.8 & 0.4 \\ -0.4 & 0.8 \end{bmatrix}, \qquad \boldsymbol{Y}_1 = \frac{1}{1.25}\begin{bmatrix} 1 \\ -0.5 \end{bmatrix}, \qquad \boldsymbol{Y}_2 = \frac{1}{1.5625}\begin{bmatrix} 0.75 \\ -1 \end{bmatrix},$$

that is $(U_1, V_1) = (0.8, -0.4)$ and $(U_2, V_2) = (0.48, -0.64)$. $U_n^2 + V_n^2$ goes $1, 0.8, 0.64$: it is divided by 1.25 at each step. Each point lies on the same ray as the forward Euler point of Example 1.1: $(0.8, -0.4) = 0.8\,(1, -0.5)$ and $(0.48, -0.64) = 0.64\,(0.75, -1)$. The true point $(0.540, -0.841)$ is 0.210 away.

Neither Euler method is safe for an oscillation

It is tempting to call backward Euler "the stable one" and stop. For a decaying problem that is often all we need. For an oscillation, damping the amplitude to zero is as wrong as blowing it up: after enough steps a bell that should still be ringing has gone silent. Stability, $|\lambda| \le 1$, is necessary, but the oscillator wants $|\lambda| = 1$.

3 Leapfrog

One more choice of grid points, mixing old and new:

$$\text{RHS} = \begin{bmatrix} V_n \\ -U_{n+1} \end{bmatrix} \quad \Longrightarrow \quad \begin{cases} U_{n+1} = U_n + h\,V_n \\ V_{n+1} = V_n - h\,U_{n+1} \end{cases}$$

Update the position first, using the old velocity; then update the velocity, using the new position. The method is *explicit* even though U_{n+1} appears on the right-hand side: by the time the second equation is used, U_{n+1} is already known. Each step costs one evaluation of the force, the same as forward Euler. For a general force, $\ddot{u} = F(u)$, the updates are $U_{n+1} = U_n + h\,V_n$ and $V_{n+1} = V_n + h\,F(U_{n+1})$; for the oscillator $F(u) = -u$.

The mixed choice of grid points looks arbitrary, but it is what comes out of the second-order equation $\ddot{u} = F(u)$ when $\ddot{u}$ is replaced by a centred difference. That derivation also explains its accuracy, its step limit and what it conserves.

The centred second difference

Write the Taylor series of u one step ahead and one step behind, with every derivative taken at t_n:

$$u(t_n + h) = u + h\,\dot{u} + \frac{h^2}{2}\,\ddot{u} + \frac{h^3}{6}\,\dddot{u} + \frac{h^4}{24}\,u^{(4)} + \frac{h^5}{120}\,u^{(5)} + \cdots,$$
$$u(t_n - h) = u - h\,\dot{u} + \frac{h^2}{2}\,\ddot{u} - \frac{h^3}{6}\,\dddot{u} + \frac{h^4}{24}\,u^{(4)} - \frac{h^5}{120}\,u^{(5)} + \cdots.$$

The two lines differ only in the signs of the odd powers of h. Add them: the terms $h\,\dot{u}$, $\frac{h^3}{6}\,\dddot{u}$ and $\frac{h^5}{120}u^{(5)}$ appear once with $+$ and once with $-$ and cancel, while the even powers are doubled:

$$u(t_n + h) + u(t_n - h) = 2u + 2\cdot\frac{h^2}{2}\,\ddot{u} + 2\cdot\frac{h^4}{24}\,u^{(4)} + \cdots = 2u(t_n) + h^2\,\ddot{u}(t_n) + \frac{h^4}{12}\,u^{(4)}(t_n) + \cdots.$$

Subtract $2u(t_n)$ from both sides:

$$u(t_{n+1}) - 2u(t_n) + u(t_{n-1}) = h^2\,\ddot{u}(t_n) + \frac{h^4}{12}\,u^{(4)}(t_n) + \cdots,$$

and divide by h^2:

$$\frac{u(t_{n+1}) - 2u(t_n) + u(t_{n-1})}{h^2} = \ddot{u}(t_n) + \frac{h^2}{12}\,u^{(4)}(t_n) + \cdots.$$

This is the centred second difference of Lecture 2, now taken in time, and its error is of order h^2. For a quadratic $u(t)$ the fourth derivative is zero and the formula has no error at all; Example 3.1 uses this. Put the centred difference in place of $\ddot{u}$ in $\ddot{u} = -u$:

$$U_{n+1} - 2U_n + U_{n-1} = -h^2\,U_n.$$

This is the two-step form of leapfrog: each new position comes from the two before it.

Velocities at half steps

To turn the two-step form into a one-step method, bring in the velocity between two positions,

$$V_{n+1/2} = \frac{U_{n+1} - U_n}{h}.$$

How good a velocity is it? Expand about the midpoint $t_{n+1/2} = t_n + h/2$, now with every derivative taken at $t_{n+1/2}$:

$$u(t_{n+1}) = u\big(t_{n+1/2} + \tfrac{h}{2}\big) = u + \frac{h}{2}\,\dot{u} + \frac{h^2}{8}\,\ddot{u} + \frac{h^3}{48}\,\dddot{u} + \cdots,$$
$$u(t_n) = u\big(t_{n+1/2} - \tfrac{h}{2}\big) = u - \frac{h}{2}\,\dot{u} + \frac{h^2}{8}\,\ddot{u} - \frac{h^3}{48}\,\dddot{u} + \cdots.$$

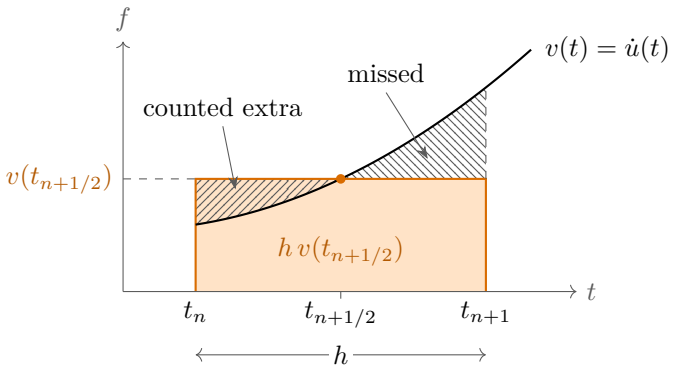
This time subtract: the even powers cancel and the odd powers double,

$$u(t_{n+1}) - u(t_n) = h\,\dot{u}(t_{n+1/2}) + \frac{h^3}{24}\,\dddot{u}(t_{n+1/2}) + \cdots,$$

and after dividing by h,

$$\frac{u(t_{n+1}) - u(t_n)}{h} = \dot{u}(t_{n+1/2}) + \frac{h^2}{24}\,\dddot{u}(t_{n+1/2}) + \cdots.$$

So $V_{n+1/2}$ is a centred difference too: it approximates the velocity at the half step with an error of order h^2. In terms of area, the drift $U_{n+1} = U_n + h\,V_{n+1/2}$ is the midpoint rule for $u(t_{n+1}) - u(t_n) = \int_{t_n}^{t_{n+1}} v\,dt$ (sketch below).



The drift as a rule for the integral of v: a rectangle whose height is taken at the midpoint, $V_{n+1/2} \approx v(t_{n+1/2})$. On the left half it counts the hatched piece above the curve, on the right half it misses the hatched piece below it. Each piece is close to a triangle of base $h/2$ and height $\frac{h}{2}\ddot{u}$, so the two cancel to first order. What is left comes from the bending of the curve: about $\frac{h^3}{24}\dddot{u}$, the term in the expansion above. The kick $V_{n+1/2} = V_{n-1/2} - h\,U_n$ is the same rule for $\int \dot{v}\,dt = -\int u\,dt$ over $[t_{n-1/2}, t_{n+1/2}]$, whose midpoint is t_n.

Now write the left side of the two-step form as a difference of two such velocities:

$$U_{n+1} - 2U_n + U_{n-1} = (U_{n+1} - U_n) - (U_n - U_{n-1}) = h\,V_{n+1/2} - h\,V_{n-1/2}.$$

The two-step form says that this equals $-h^2 U_n$, and dividing by h,

$$V_{n+1/2} = V_{n-1/2} - h\,U_n.$$

So the two-step form has split into

$$U_{n+1} = U_n + h\,V_{n+1/2}, \qquad V_{n+1/2} = V_{n-1/2} - h\,U_n.$$

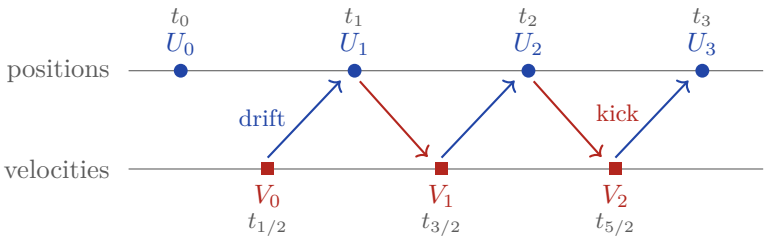
The first equation is the definition of $V_{n+1/2}$, and the second is the two-step form divided by h. Shift the second equation by one step ($n \to n+1$),

$$V_{n+3/2} = V_{n+1/2} - h\,U_{n+1},$$

so that both updates go from step n to step $n+1$ in the order they are computed: first the new position U_{n+1}, then the velocity after it. Finally write V_n for $V_{n+1/2}$, the velocity half a step after t_n (and V_{n+1} for $V_{n+3/2}$):

$$U_{n+1} = U_n + h\,V_n, \qquad V_{n+1} = V_n - h\,U_{n+1}.$$

These are the leapfrog updates at the start of this section. Positions live at whole steps and velocities at half steps, and each update leaps over the other variable:



Start with half a kick

V_0 stands for the velocity at $t = h/2$. Expand v about $t = 0$ and use $\dot{v} = \ddot{u} = -u$:

$$v(\tfrac{h}{2}) = v(0) + \frac{h}{2}\,\dot{v}(0) + \frac{h^2}{8}\,\ddot{v}(0) + \cdots = v(0) - \frac{h}{2}\,u(0) + \mathcal{O}(h^2).$$

So start with half a step of the force $-u$:

$$V_0 = v(0) - \tfrac{h}{2}\,u(0).$$

With $V_0 = v(0)$ the method still runs and stays stable, but its error drops to first order. With 100 steps per period the largest position error over one period is 0.031 that way, against 0.00079 with the half kick. For a general force the half kick is $V_0 = v(0) + \frac{h}{2}\,F(u(0))$.

Example 3.1 (A thrown ball)**. Given.** A ball thrown upwards, $\ddot{u} = -g$ with $g = 10$ m/s^2, $u(0) = 0$, $v(0) = 10$ m/s, stepped with leapfrog at $h = 0.5$ s and the half kick. The true height is $u(t) = 10t - 5t^2$.

Find. What are V_0 and $U_1, \ldots, U_4$? How do they compare with $u(t)$? What does forward Euler give with the same step?

Answer. The force is $F = -g$, so the half kick gives $V_0 = 10 - 0.25 \cdot 10 = 7.5$, and each step adds $h\,V_n$ to the position and $-hg = -5$ to the velocity:

$$\begin{aligned} U_1 &= 0 + 0.5 \cdot 7.5 = 3.75, & V_1 &= 7.5 - 5 = 2.5,\\ U_2 &= 3.75 + 0.5 \cdot 2.5 = 5, & V_2 &= 2.5 - 5 = -2.5,\\ U_3 &= 5 + 0.5 \cdot (-2.5) = 3.75, & V_3 &= -2.5 - 5 = -7.5,\\ U_4 &= 3.75 + 0.5 \cdot (-7.5) = 0. \end{aligned}$$

The true heights are $u(0.5) = 3.75$, $u(1) = 5$, $u(1.5) = 3.75$ and $u(2) = 0$: leapfrog has no error at all. The half kick is exact here, because $v = 10 - 10t$ is linear in t: $V_0 = v(h/2)$, so $U_1 = u(h)$. From then on the centred second difference, which is exact for a quadratic, keeps every U_n exact, and each V_n is the true velocity at the half step (7.5 at $t = 0.25$, 2.5 at $t = 0.75$, and so on). Forward Euler makes the same two updates, $U_{n+1} = U_n + hV_n$ and $V_{n+1} = V_n - hg$ (for a constant force the old and new positions give the same force), but it starts from $V_0 = v(0) = 10$, the velocity at $t = 0$ rather than at $h/2$. It gives $U = 5, 7.5, 7.5, 5$ at $t = 0.5, 1, 1.5, 2$: one step puts it $gh^2/2 = 1.25$ m too high, the error grows by that much at every step, and after n steps it is $1.25\,n$ m, an error of $gh\,t/2$. It lands 5 m above the ground when the ball is back at $u = 0$. For a constant force the half kick is the whole difference.

The growth matrix

Back to the oscillator. Put the first update into the second:

$$V_{n+1} = V_n - h\,U_{n+1} = V_n - h\,(U_n + hV_n) = -h\,U_n + (1 - h^2)\,V_n.$$

So

$$\boldsymbol{G}_L = \begin{bmatrix} 1 & h \\ -h & 1 - h^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -h & 1 \end{bmatrix} \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix},$$

a *drift* (the position moves with the velocity) followed by a *kick* (the velocity changes with the force). Each factor has determinant 1, so $\det \boldsymbol{G}_L = 1$ (directly, $1 \cdot (1 - h^2) - h \cdot (-h) = 1$), and $\operatorname{tr} \boldsymbol{G}_L = 2 - h^2$.

Eigenvalues and the step limit

The characteristic polynomial is

$$\det(\boldsymbol{G}_L - \lambda \boldsymbol{I}) = (1-\lambda)(1-h^2-\lambda) + h^2 = \lambda^2 - (2-h^2)\,\lambda + 1.$$

The two-step form gives the same polynomial: put $U_n = \lambda^n$ into $U_{n+1} - 2U_n + U_{n-1} = -h^2 U_n$ and divide by λ^{n-1} to get $\lambda^2 - 2\lambda + 1 = -h^2\lambda$. The roots are

$$\lambda = \frac{(2-h^2) \pm \sqrt{(2-h^2)^2 - 4}}{2}, \qquad (2-h^2)^2 - 4 = h^2(h^2-4),$$

with product 1 and sum $2 - h^2$. There are three cases.

- $h < 2$: the square root is imaginary, and $\lambda = 1 - \frac{h^2}{2} \pm ih\sqrt{1 - \frac{h^2}{4}}$ are complex conjugates with

$$|\lambda|^2 = \Big(1 - \frac{h^2}{2}\Big)^2 + h^2\Big(1 - \frac{h^2}{4}\Big) = 1 - h^2 + \frac{h^4}{4} + h^2 - \frac{h^4}{4} = 1.$$

  For every $h < 2$ the computed oscillation neither grows nor decays.

- $h = 2$: the double root $\lambda = -1$. The solutions are $U_n = (c + dn)(-1)^n$, which grow in proportion to n.

- $h > 2$: the roots are real, with product 1 and sum $2 - h^2 < -2$. Both are negative and one of them is below -1, so the method blows up.

The step limit $h < 2$ means more than π steps per period. Inside the limit $\lambda + \bar{\lambda} = 2\cos\theta = 2 - h^2$, so

$$\cos\theta = 1 - \frac{h^2}{2}, \qquad 2\sin^2\frac{\theta}{2} = 1 - \cos\theta = \frac{h^2}{2}, \qquad \theta = 2\arcsin\frac{h}{2} = h + \frac{h^3}{24} + \cdots,$$

a little more than h: the computed oscillation runs slightly ahead of the true one.

For the masses and springs of last time, $\ddot{\boldsymbol{u}} + \boldsymbol{K}\boldsymbol{u} = \boldsymbol{0}$ with $\boldsymbol{M} = \boldsymbol{I}$, write $\boldsymbol{K} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{\mathsf{T}}$ and $\boldsymbol{u} = \boldsymbol{Q}\boldsymbol{x}$ as in Lecture 13. Each normal mode obeys $\ddot{x}_j + \omega_j^2 x_j = 0$, where ω_j^2 is the jth eigenvalue of $\boldsymbol{K}$. Leapfrog is linear, so in these coordinates it acts on each x_j separately, and on x_j it is leapfrog on our oscillator with the step $h\omega_j$ (rescale the velocity by ω_j). So it is stable when

$$h\,\omega_{\max} < 2.$$

The fastest mode sets the step, even when it carries almost none of the motion. Molecular dynamics codes use leapfrog (in the form called the Verlet method), and their steps of one or two femtoseconds come from the fastest motion: a bond to a hydrogen atom vibrates with a period near 10 fs, which puts the stability limit near 3 fs, and accuracy asks for less. Explicit crash simulations of cars are limited in the same way by their smallest and stiffest elements.

What leapfrog conserves

For the true motion, multiplying $\ddot{u} + u = 0$ by $\dot{u}$ gives $\frac{1}{2}\frac{d}{dt}(\dot{u}^2 + u^2) = 0$: the energy is constant. Do the same to the two-step form, with $U_{n+1} - U_{n-1}$ ($2h$ times a centred velocity) in the role of $\dot{u}$:

$$(U_{n+1} - 2U_n + U_{n-1})(U_{n+1} - U_{n-1}) = -h^2\,U_n(U_{n+1} - U_{n-1}).$$

On the left, $U_{n+1} - 2U_n + U_{n-1} = (U_{n+1} - U_n) - (U_n - U_{n-1})$ and $U_{n+1} - U_{n-1} = (U_{n+1} - U_n) + (U_n - U_{n-1})$, so the product is a difference of squares,

$$(U_{n+1} - U_n)^2 - (U_n - U_{n-1})^2 = h^2(V_{n+1/2}^2 - V_{n-1/2}^2).$$

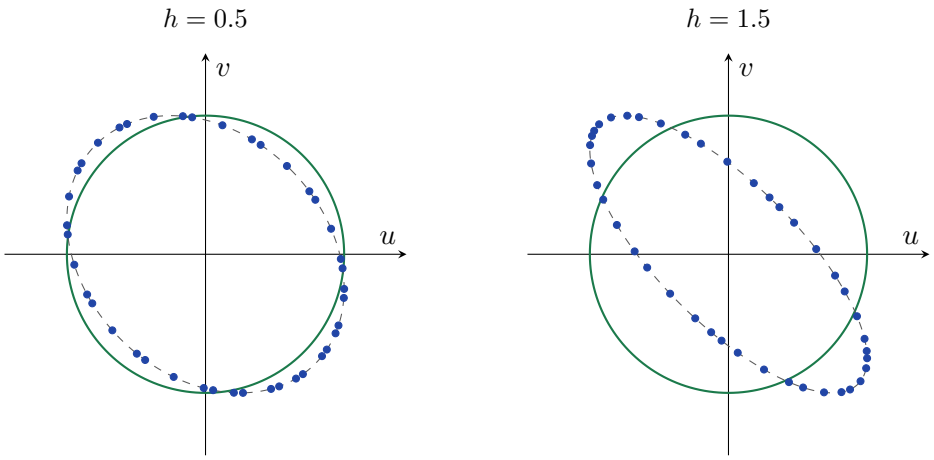
The right side is $-h^2(U_nU_{n+1} - U_{n-1}U_n)$. Divide by h^2 and collect:

$$V_{n+1/2}^2 + U_nU_{n+1} = V_{n-1/2}^2 + U_{n-1}U_n.$$

So this quantity does not change from step to step. In the notation of the updates (V_n for $V_{n+1/2}$, and $U_{n+1} = U_n + hV_n$) it is

$$Q_n = U_nU_{n+1} + V_n^2 = U_n^2 + h\,U_nV_n + V_n^2.$$

Leapfrog does not keep $U^2 + V^2$ ($\boldsymbol{G}_L$ is not orthogonal), but it keeps this nearby quantity. The points (U_n, V_n) lie on the ellipse $U^2 + hUV + V^2 = Q_0$: a circle squeezed along the diagonal $u = v$ and stretched along $u = -v$, which becomes the circle as $h \to 0$. The energy error stays bounded however many periods we compute; it does not drift.



Leapfrog from $u(0) = 1$, $v(0) = 0$ with half a kick, 40 steps. The points stay on the dashed ellipse $U^2 + hUV + V^2 = 1 - h^2/4$; the true solution stays on the green circle of radius 1. As h approaches 2 the ellipse flattens towards the line $u = -v$.

Example 3.2 (Leapfrog by hand)**. Given.** $\ddot{u} + u = 0$ with $u(0) = 1$, $v(0) = 0$, the step $h = 0.5$, and the half-kick start.

Find. What is $\boldsymbol{Y}_0$? What are $\boldsymbol{Y}_1 = \boldsymbol{G}_L\boldsymbol{Y}_0$ and $\boldsymbol{Y}_2 = \boldsymbol{G}_L\boldsymbol{Y}_1$? Does $U_n^2 + V_n^2$ stay fixed? Does Q_n?

Answer. With h as a letter, $\boldsymbol{Y}_0 = (1, -h/2)$ and

$$\boldsymbol{Y}_1 = \boldsymbol{G}_L\boldsymbol{Y}_0 = \begin{bmatrix} 1 & h \\ -h & 1 - h^2 \end{bmatrix} \begin{bmatrix} 1 \\ -h/2 \end{bmatrix} = \begin{bmatrix} 1 - h^2/2 \\ -h - (1 - h^2)\,h/2 \end{bmatrix} = \begin{bmatrix} 1 - h^2/2 \\ -(3h - h^3)/2 \end{bmatrix}.$$

Now put $h = 0.5$:

$$\boldsymbol{G}_L = \begin{bmatrix} 1 & 0.5 \\ -0.5 & 0.75 \end{bmatrix}, \qquad \boldsymbol{Y}_0 = \begin{bmatrix} 1 \\ -0.25 \end{bmatrix}, \qquad \boldsymbol{Y}_1 = \begin{bmatrix} 0.875 \\ -0.6875 \end{bmatrix},$$

$$\boldsymbol{Y}_2 = \boldsymbol{G}_L \boldsymbol{Y}_1 = \begin{bmatrix} 0.875 - 0.34375 \\ -0.4375 - 0.515625 \end{bmatrix} = \begin{bmatrix} 0.53125 \\ -0.953125 \end{bmatrix}.$$

Step by step this is $U_1 = 1 + 0.5\,(-0.25) = 0.875$, $V_1 = -0.25 - 0.5\,(0.875) = -0.6875$, $U_2 = 0.875 + 0.5\,(-0.6875) = 0.53125$ and $V_2 = -0.6875 - 0.5\,(0.53125) = -0.953125$. U_2 should be compared with $u(1) = 0.540$ and V_2 with the velocity at the half step, $v(1.25) = -\sin 1.25 = -0.949$: together they are off by 0.010. The sum $U_n^2 + V_n^2$ goes $1.0625, 1.2383, 1.1907$ and wanders. But $Q_n = U_n^2 + 0.5\,U_nV_n + V_n^2 = 0.9375$ at every step, which is $1 - h^2/4$. The two-step form holds as well: $U_2 - 2U_1 + U_0 = -0.21875 = -h^2U_1$.

Example 3.3 (Above the step limit)**. Given.** Leapfrog for $\ddot{u} + u = 0$ with $h = 2.5$.

Find. What are the eigenvalues of $\boldsymbol{G}_L$? What happens to the computed solution?

Answer. $\operatorname{tr} \boldsymbol{G}_L = 2 - 6.25 = -4.25$ and $\det \boldsymbol{G}_L = 1$, so $\lambda^2 + 4.25\,\lambda + 1 = 0$ and $\lambda = -4$ or $\lambda = -\frac{1}{4}$. For $\lambda = -4$,

$$\boldsymbol{G}_L + 4\boldsymbol{I} = \begin{bmatrix} 5 & 2.5 \\ -2.5 & -1.25 \end{bmatrix} \qquad \text{has null vector} \qquad \boldsymbol{x} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

The part of $\boldsymbol{Y}_0$ along $\boldsymbol{x}$ is multiplied by -4 at every step: after ten steps by $4^{10} \approx 10^6$, with the sign changing at each step. Nothing in the code signals the problem; the numbers grow.

4 The trapezoidal method

The set-up again

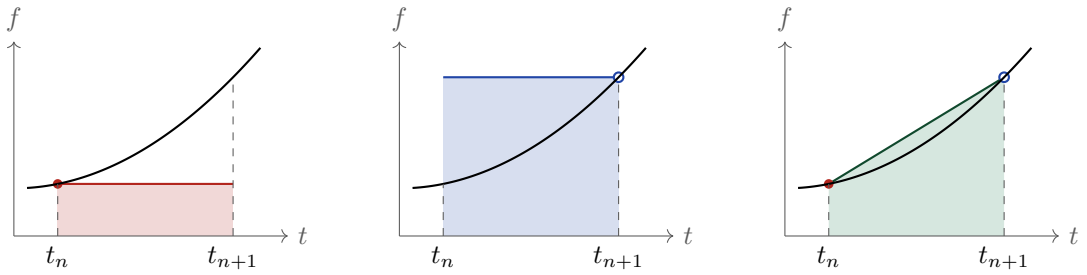
Recall the notation. The step is h and $t_n = nh$. The vector $\boldsymbol{Y}_n = (U_n, V_n)$ is the computed approximation to $\boldsymbol{y}(t_n) = (u(t_n), v(t_n))$. The oscillator is $\dot{\boldsymbol{y}} = \boldsymbol{f}(t, \boldsymbol{y})$ with $\boldsymbol{f} = \boldsymbol{A}\boldsymbol{y} = (v, -u)$ and $\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ -1 & 0 \end{smallmatrix}\right]$. Over one step the exact solution satisfies

$$\boldsymbol{y}(t_{n+1}) = \boldsymbol{y}(t_n) + \int_{t_n}^{t_{n+1}} \boldsymbol{f}\big(t, \boldsymbol{y}(t)\big)\, dt.$$

Forward Euler replaced the integral by the rectangle $h\,\boldsymbol{f}(t_n, \boldsymbol{Y}_n)$, with the height at the left end. Backward Euler used $h\,\boldsymbol{f}(t_{n+1}, \boldsymbol{Y}_{n+1})$, the height at the right end. The trapezoidal method uses the trapezoid, the average of the two heights:

$$\boldsymbol{Y}_{n+1} = \boldsymbol{Y}_n + \frac{h}{2}\Big(\boldsymbol{f}(t_n, \boldsymbol{Y}_n) + \boldsymbol{f}(t_{n+1}, \boldsymbol{Y}_{n+1})\Big).$$

Its right-hand side is the average of the forward and backward Euler right-hand sides, and like backward Euler it is implicit.



Three rules for the integral of $\boldsymbol{f}$ over one step (shaded), against the area under the curve. For the rising, curving f drawn here the left rectangle is too small, the right one too large, and the trapezoid lies between them.

Why the average is better

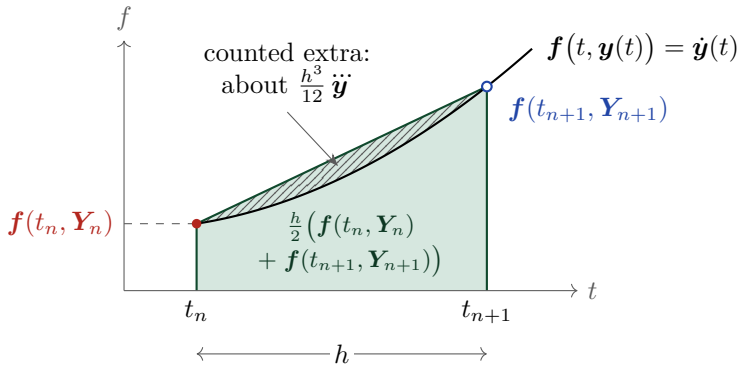
Taylor's theorem compares the difference quotient with the average of the two end slopes. Expand $u(t_{n+1})$ and $\dot{u}(t_{n+1})$ about t_n (every derivative at t_n):

$$\frac{u(t_{n+1}) - u(t_n)}{h} = \dot{u} + \frac{h}{2}\,\ddot{u} + \frac{h^2}{6}\,\dddot{u} + \cdots,$$
$$\frac{\dot{u}(t_n) + \dot{u}(t_{n+1})}{2} = \frac{\dot{u} + (\dot{u} + h\,\ddot{u} + \frac{h^2}{2}\,\dddot{u} + \cdots)}{2} = \dot{u} + \frac{h}{2}\,\ddot{u} + \frac{h^2}{4}\,\dddot{u} + \cdots.$$

The first two terms agree, and the difference starts at h^2:

$$\frac{u(t_{n+1}) - u(t_n)}{h} = \frac{\dot{u}(t_n) + \dot{u}(t_{n+1})}{2} - \frac{h^2}{12}\,\dddot{u}(t_n) + \cdots.$$

The difference quotient matches the average of the two end slopes with an error of order h^2, one order better than either end alone (the $\pm\frac{h}{2}\ddot{u}$ errors of the two Euler methods cancel). So the trapezoidal method is second-order accurate. The sketch below shows the same cancellation with areas.



The trapezoidal method as a rule for the integral, on the same curve as the two Euler sketches. The top of the trapezoid is the chord from the old value (filled dot) to the new one (open circle). Its area is the average of the forward and backward Euler rectangles, so their errors, about $\frac{h^2}{2}\ddot{\boldsymbol{y}}$ missed and $\frac{h^2}{2}\ddot{\boldsymbol{y}}$ counted extra, cancel. What is left is the thin hatched piece between the chord and the curve, counted extra here because the curve bends upwards: about $\frac{h^3}{12}\,\dddot{\boldsymbol{y}}$, one order smaller.

The trapezoidal method on the oscillator

Evaluate the right-hand side as the *average* of the old and new time levels:

$$\text{RHS} = \begin{bmatrix} \dfrac{V_n + V_{n+1}}{2} \\ -\dfrac{U_n + U_{n+1}}{2} \end{bmatrix} \qquad \Longrightarrow \qquad \begin{cases} U_{n+1} = U_n + \frac{h}{2}\,(V_n + V_{n+1}) \\ V_{n+1} = V_n - \frac{h}{2}\,(U_n + U_{n+1}) \end{cases}$$

The energy first. Write the first equation as $U_{n+1} - U_n = \frac{h}{2}\,(V_n + V_{n+1})$ and multiply it by $U_n + U_{n+1}$; write the second as $V_{n+1} - V_n = -\frac{h}{2}\,(U_n + U_{n+1})$ and multiply it by $V_n + V_{n+1}$:

$$U_{n+1}^2 - U_n^2 = \tfrac{h}{2}\,(V_n + V_{n+1})(U_n + U_{n+1}), \qquad V_{n+1}^2 - V_n^2 = -\tfrac{h}{2}\,(U_n + U_{n+1})(V_n + V_{n+1}).$$

The right sides cancel when added, so $U_{n+1}^2 + V_{n+1}^2 = U_n^2 + V_n^2$ at every step and for every h. This is the discrete copy of $\frac{d}{dt}(u^2 + v^2) = 2uv + 2v(-u) = 0$, with u and v replaced by the averages of old and new values.

For the growth matrix, collect the new time level on the left and the old on the right:

$$\begin{cases} U_{n+1} - \frac{h}{2}\,V_{n+1} = U_n + \frac{h}{2}\,V_n \\ \frac{h}{2}\,U_{n+1} + V_{n+1} = V_n - \frac{h}{2}\,U_n \end{cases} \qquad \text{that is} \qquad \Big(\boldsymbol{I} - \frac{h}{2}\boldsymbol{A}\Big)\boldsymbol{Y}_{n+1} = \Big(\boldsymbol{I} + \frac{h}{2}\boldsymbol{A}\Big)\boldsymbol{Y}_n,$$

so

$$\boldsymbol{G}_T = \begin{bmatrix} 1 & -\frac{h}{2} \\ \frac{h}{2} & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & \frac{h}{2} \\ -\frac{h}{2} & 1 \end{bmatrix} = \frac{1}{1 + \frac{h^2}{4}} \begin{bmatrix} 1 & \frac{h}{2} \\ -\frac{h}{2} & 1 \end{bmatrix}^2 = \frac{1}{1 + \frac{h^2}{4}} \begin{bmatrix} 1 - \frac{h^2}{4} & h \\ -h & 1 - \frac{h^2}{4} \end{bmatrix}.$$

The middle step used the fact that the inverse of $\left[\begin{smallmatrix} 1 & -h/2 \\ h/2 & 1 \end{smallmatrix}\right]$ is $\frac{1}{1+h^2/4}\left[\begin{smallmatrix} 1 & h/2 \\ -h/2 & 1 \end{smallmatrix}\right]$, a multiple of the matrix it is being multiplied by.

The columns of the result confirm the energy argument. Each has squared length

$$\frac{\left(1 - \frac{h^2}{4}\right)^2 + h^2}{\left(1 + \frac{h^2}{4}\right)^2} = \frac{1 - \frac{h^2}{2} + \frac{h^4}{16} + h^2}{\left(1 + \frac{h^2}{4}\right)^2} = \frac{\left(1 + \frac{h^2}{4}\right)^2}{\left(1 + \frac{h^2}{4}\right)^2} = 1,$$

and the two columns are orthogonal. So $\boldsymbol{G}_T$ is an **orthogonal matrix**, a pure rotation, and $\boldsymbol{Y}_n = \boldsymbol{G}_T^n \boldsymbol{Y}_0$ keeps the length of $\boldsymbol{Y}_0$ for every h.

Both factors of $\boldsymbol{G}_T$ are built from $\boldsymbol{A}$, so they share its eigenvectors, and an eigenvalue $\mu = \pm i$ of $\boldsymbol{A}$ becomes $(1 + h\mu/2)/(1 - h\mu/2)$:

$$\lambda = \frac{1 \pm ih/2}{1 \mp ih/2}, \qquad |\lambda| = 1, \qquad \theta = \arg\Big(1 + \frac{ih}{2}\Big) - \arg\Big(1 - \frac{ih}{2}\Big) = 2\arctan\frac{h}{2} = h - \frac{h^3}{12} + \cdots.$$

The numerator and denominator are complex conjugates, which is why $|\lambda| = 1$. The computed oscillation keeps its amplitude and falls slightly behind, at twice the rate at which leapfrog runs ahead. The cost is one solve per step, as for backward Euler.

The order shows in the growth matrix too. Write $\boldsymbol{X} = \frac{h}{2}\boldsymbol{A}$ and expand the inverse in a series (for small h):

$$\begin{aligned} (\boldsymbol{I} - \boldsymbol{X})^{-1} &= \boldsymbol{I} + \boldsymbol{X} + \boldsymbol{X}^2 + \boldsymbol{X}^3 + \cdots, \\ \boldsymbol{G}_T &= (\boldsymbol{I} + \boldsymbol{X} + \boldsymbol{X}^2 + \boldsymbol{X}^3 + \cdots)(\boldsymbol{I} + \boldsymbol{X}) = \boldsymbol{I} + 2\boldsymbol{X} + 2\boldsymbol{X}^2 + 2\boldsymbol{X}^3 + \cdots \\ &= \boldsymbol{I} + h\boldsymbol{A} + \frac{(h\boldsymbol{A})^2}{2} + \frac{(h\boldsymbol{A})^3}{4} + \cdots, \end{aligned}$$

which agrees with $e^{h\boldsymbol{A}} = \boldsymbol{I} + h\boldsymbol{A} + (h\boldsymbol{A})^2/2 + (h\boldsymbol{A})^3/6 + \cdots$ up to the square and differs by $(h\boldsymbol{A})^3/12$: second order again. Applied to the heat equation the trapezoidal method is called the Crank–Nicolson method. It is also the default integration method of SPICE circuit simulators.

Example 4.1 (The trapezoidal method by hand)**. Given.** $\ddot{u}+u=0$ with $u(0)=1$, $v(0)=0$, and $h=0.5$.

Find. What is $\boldsymbol{G}_T$? What are (U_1,V_1) and (U_2,V_2)? How does the angle per step compare with h?

Answer. With h as a letter, $\boldsymbol{Y}_1=\boldsymbol{G}_T\boldsymbol{Y}_0=\dfrac{1}{1+h^2/4}(1-h^2/4,\ -h)$, the first column of $\boldsymbol{G}_T$. For $h=0.5$, $1+h^2/4=17/16$ and $1-h^2/4=15/16$, so

$$\boldsymbol{G}_T=\frac{16}{17}\begin{bmatrix}\frac{15}{16} & \frac{1}{2}\\ -\frac{1}{2} & \frac{15}{16}\end{bmatrix}=\frac{1}{17}\begin{bmatrix}15 & 8\\ -8 & 15\end{bmatrix}.$$

The columns are unit vectors because $8^2+15^2=17^2$. Then

$$(U_1,V_1)=\left(\tfrac{15}{17},-\tfrac{8}{17}\right)=(0.882,-0.471),\qquad (U_2,V_2)=\left(\tfrac{161}{289},-\tfrac{240}{289}\right)=(0.557,-0.830),$$

and $U_n^2+V_n^2=1$ at every step ($161^2+240^2=289^2$). The angle per step is $\arctan(8/15)=0.4900$ against $h=0.5$, behind by 0.0100 per step, close to $h^3/12=0.0104$. The true point at $t=1$ is 0.020 away, and all of that is phase error.

Structure, not just accuracy

Forward Euler grows, backward Euler decays, the trapezoidal method does neither. The reason is not that the trapezoidal method is more accurate in a Taylor-series sense. It is that its growth matrix is orthogonal, and orthogonal matrices preserve length. Leapfrog, which is explicit, keeps a nearby quadratic quantity instead. Choosing a scheme whose discrete operator inherits the structure of the continuous one (here, conservation of energy) is a different design principle from chasing one more order of h, and often the more useful one.

Fast decaying modes ring

For a fast decay $\dot{y}=-ay$ the trapezoidal method multiplies by $(1-ah/2)/(1+ah/2)$ at each step, which is close to -1 when ah is large. So it barely damps such a mode: it changes the sign of the mode at every step and shrinks it only a little. Backward Euler multiplies by $1/(1+ah)$, close to 0, and removes the mode within a step or two. In circuit simulators that use the trapezoidal method this shows up as ringing after a sudden switch.

5 The four methods so far

Each method replaces the integral of $\boldsymbol{f}$ over one step by an area, and each area misses part of the true one. The examples of this lecture all take two steps of $h=0.5$ from $(u,v)=(1,0)$.

method	right-hand side at		order	$\lvert\lambda\rvert$ for $\ddot{u}+u=0$	two steps: off by
forward Euler	t_n	explicit	1	$\sqrt{1+h^2}$	0.263 (Example 1.1)
backward Euler	t_{n+1}	implicit	1	$1/\sqrt{1+h^2}$	0.210 (Example 2.1)
leapfrog	half steps	explicit	2	1 for $h<2$	0.010 (Example 3.2)
trapezoidal	average of both ends	implicit	2	1	0.020 (Example 4.1)

Leapfrog's distance uses its velocity half a step later, as in Example 3.2.

In the area sketches, one step of the integral of f (with f along the true solution, so $\dot{f} = \ddot{y}$ and $\ddot{f} = \dddot{y}$) has these errors:

rule	area	what one step misses	used by
left rectangle	$h\,f(t_n)$	$+\frac{h^2}{2}\,\dot{f}$	forward Euler
right rectangle	$h\,f(t_{n+1})$	$-\frac{h^2}{2}\,\dot{f}$	backward Euler
midpoint	$h\,f(t_{n+1/2})$	$+\frac{h^3}{24}\,\ddot{f}$	leapfrog
trapezoid	$\frac{h}{2}(f(t_n)+f(t_{n+1}))$	$-\frac{h^3}{12}\,\ddot{f}$	trapezoidal

A plus sign means the rule gives too little, a minus sign too much. The two second-order rules err in opposite directions, and the trapezoid errs twice as much as the midpoint rule.

Where this is going

Next time we take the last two rows of the table further. A weighted average of the midpoint and trapezoid areas cancels their h^3 errors; the result is Simpson's rule, which is fourth order. Turning Simpson's rule into a time stepper for $\dot{\boldsymbol{y}} = \boldsymbol{f}(t, \boldsymbol{y})$ gives the fourth-order Runge–Kutta method (RK4). Then we compare all five methods: accuracy for the work, stability regions, and problems with fast decaying modes.

Reading

Strang, *Computational Science and Engineering*, § 2.2 (Oscillation by Newton's Law) for leapfrog and the trapezoidal method, and § 6.2 (Finite Difference Methods for ODEs) for Runge–Kutta methods and stability. For why leapfrog keeps the energy close over long times: E. Hairer, C. Lubich and G. Wanner, Geometric numerical integration illustrated by the Störmer–Verlet method, *Acta Numerica* 12 (2003), 399–450.

Appendix: the four methods in Python

The script below builds the four growth matrices, repeats the hand calculations of Examples 1.1 to 4.1, checks leapfrog's conserved quantity and its step limit, and follows the thrown ball of Example 3.1. It needs only NumPy.

```python
"""Lecture 14: four time-stepping methods for u'' + u = 0, as y' = Ay with y = (u, v)."""
import numpy as np

A = np.array([[0.0, 1.0], [-1.0, 0.0]])
I = np.eye(2)
methods = ["forward Euler", "backward Euler", "leapfrog", "trapezoidal"]

def growth_matrix(method, h):
    if method == "forward Euler":
        return I + h * A
    if method == "backward Euler":
        return np.linalg.inv(I - h * A)    # a real code solves with I - hA instead
    if method == "leapfrog":
        return np.array([[1.0, h], [-h, 1.0 - h * h]])
    if method == "trapezoidal":
```

```
        return np.linalg.solve(I - h * A / 2, I + h * A / 2)

def start(method, h):            # u(0) = 1, v(0) = 0; leapfrog gets half a kick
    return np.array([1.0, -h / 2 if method == "leapfrog" else 0.0])

def distance(method, y, t, h):   # leapfrog's V is the velocity half a step later
    s = h / 2 if method == "leapfrog" else 0.0
    return np.hypot(y[0] - np.cos(t), y[1] + np.sin(t + s))

# 1. two steps of h = 0.5, and the size and angle of the eigenvalues of G
h = 0.5
print("two steps of h = 0.5 from u = 1, v = 0:")
for m in methods:
    G = growth_matrix(m, h)
    y = G @ G @ start(m, h)
    lam = np.linalg.eigvals(G)[0]
    print(f"  {m:14s} U2 {y[0]:8.5f}  V2 {y[1]:8.5f}  off {distance(m, y, 1.0, h):.5f}"
          f"  |lambda| {abs(lam):.5f}  angle {abs(np.angle(lam)):.5f}")

# 2. leapfrog keeps Q = U^2 + hUV + V^2, and fails above h = 2
y = start("leapfrog", h)
Q = []
for n in range(6):
    Q.append(y[0] ** 2 + h * y[0] * y[1] + y[1] ** 2)
    y = growth_matrix("leapfrog", h) @ y
print("leapfrog Q at h = 0.5:", np.round(Q, 12))
print("leapfrog eigenvalues at h = 2.5:",
      np.sort(np.linalg.eigvals(growth_matrix("leapfrog", 2.5)).real))

# 2b. leapfrog on a thrown ball, u'' = -g: with the half kick it has no error at all
g, hb = 10.0, 0.5
U, V = 0.0, 10.0 - hb / 2 * g            # V0 = v(0) - (h/2) g
for n in range(1, 5):
    U = U + hb * V                        # drift
    V = V - hb * g                        # kick
    t = n * hb
    print(f"  ball t = {t:.1f}  U = {U:5.2f}  true {10 * t - 5 * t * t:5.2f}")
```

Its output:

```
two steps of h = 0.5 from u = 1, v = 0:
  forward Euler  U2  0.75000  V2 -1.00000  off 0.26288  |lambda| 1.11803  angle 0.46365
  backward Euler U2  0.48000  V2 -0.64000  off 0.21030  |lambda| 0.89443  angle 0.46365
  leapfrog       U2  0.53125  V2 -0.95312  off 0.00995  |lambda| 1.00000  angle 0.50536
  trapezoidal    U2  0.55709  V2 -0.83045  off 0.02009  |lambda| 1.00000  angle 0.48996
leapfrog Q at h = 0.5: [0.9375 0.9375 0.9375 0.9375 0.9375 0.9375]
leapfrog eigenvalues at h = 2.5: [-4.   -0.25]
  ball t = 0.5  U =  3.75  true  3.75
  ball t = 1.0  U =  5.00  true  5.00
  ball t = 1.5  U =  3.75  true  3.75
  ball t = 2.0  U =  0.00  true  0.00
```